

Tanaka Certificates: Weekly Research Report

Filip Morawiec

6 July 2026

Abstract

This week I contributed by implementing a multidimensional verifier for ReLU neural networks (with last layer being piecewise quadratic to ensure completeness) and beginning to prove its correctness. After meeting with Greg, he recommended that I should focus on proving the correctness of the ReLU NN region discovery algorithm and algebraically working through an example 2D SDE verification, which I initially did and present here. I also worked on training a certificate for this problem - following the techniques used by Greg in his previous paper - but I failed to satisfy all the constraints a certificate should have.

Contents

1	Multidimensional verifier	2
1.1	Problem setup	2
2	1D ReLU NN region discovery	3
3	Multi-dimensional ReLU NN region discovery	4
4	2D SDE verification example	7
5	Certificate training	8

1 Multidimensional verifier

1.1 Problem setup

Consider a multi-dimensional stochastic differential equation

$$dX_t = f(X_t) dt + g(X_t) dW_t \quad (1)$$

on a domain $D \subset \mathbb{R}^d$, where $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the drift, $g : \mathbb{R}^d \rightarrow \mathbb{R}^{d \times m}$ is the diffusion, and W_t is an m -dimensional Brownian motion. Write $a(x) = g(x)g(x)^\top$ for the diffusion covariance. For a reach-avoid problem, let $X_0 \subseteq D$ be the initial set, $X_u \subseteq D$ the unsafe set, and $X_\tau \subseteq D$ the target. We seek a certificate V and constants $\alpha < \beta$ and $\varepsilon > 0$ such that

1. boundary condition holds:

$$\sup_{x \in X_0} V(x) \leq \alpha \quad \text{and} \quad \inf_{x \in X_u} V(x) \geq \beta \quad (2)$$

2. generator condition holds:

$$\mathcal{L}V(x) \leq -\varepsilon \quad \text{for } x \in \{V \leq \beta\} \setminus X_\tau, \quad (3)$$

where, at smooth points,

$$\mathcal{L}V(x) = \nabla V(x) \cdot f(x) + \frac{1}{2} \text{tr}(a(x) \nabla^2 V(x)). \quad (4)$$

Following the piecewise-quadratic setup, suppose that a compact verification region $K \subseteq D$ is partitioned into finitely many polyhedral cells,

$$K = \bigcup_{i=1}^N K_i,$$

and that V is continuous across their interfaces and quadratic on each cell:

$$V(x) = V_i(x) = c_i + p_i^\top x + \frac{1}{2} x^\top Q_i x, \quad x \in K_i, \quad Q_i = Q_i^\top. \quad (5)$$

Consequently, inside K_i ,

$$\mathcal{L}V_i(x) = (p_i + Q_i x) \cdot f(x) + \frac{1}{2} \text{tr}(a(x) Q_i). \quad (6)$$

Besides the interior generator condition, the multidimensional Itô–Tanaka formula introduces a surface-local-time term on cell interfaces. For every shared face F between K_i and K_j , with unit normal $n_{i \rightarrow j, F}$ pointing from K_i to K_j , this contribution is nonpositive when

$$(\nabla V_j(x) - \nabla V_i(x)) \cdot n_{i \rightarrow j, F} = (p_j - p_i + (Q_j - Q_i)x) \cdot n_{i \rightarrow j, F} \leq 0, \quad x \in F. \quad (7)$$

Together with square-integrability of the stochastic integral, continuity, the cellwise generator condition, and this interface condition ensure that $V(X_t)$ is a supermartingale up to the first exit time from K .

The process is stopped after reaching the target or unsafe set: once the reach-avoid property has been achieved or violated, its later trajectory is irrelevant.

Remark. We model a certificate V as:

$$V(x) = \sum_{k=1}^m \lambda_k \phi(z_k(x)) + c, \quad (8)$$

$$z(x) = \text{NN}(x) = W_L \text{ReLU}(\dots \text{ReLU}(W_1 x + b_1) \dots) + b_L, \quad (9)$$

$$\phi = \text{PiecewiseQuadratic1D}. \quad (10)$$

This way, because we include a piecewise quadratic function ϕ in the last layer, we cover a certain failure case related to the curvature term $V''(x)$ in the generator condition (3).

2 1D ReLU NN region discovery

Consider the following Algorithm 1 which discovers the linear cells of a one-dimensional ReLU network.

Listing 1: Discover one-dimensional linear cells of a ReLU network

```

1 def discover_cells(layers):
2     """
3     The network has a one-dimensional input and a one-dimensional output, but hidden
4     layers can be multi-dimensional:
5     >>> layers[i] = (W_i, b_i)
6     >>> W_i.shape = (n_{i+1}, n_i)
7     >>> b_i.shape = (n_{i+1},)
8     """
9
10    # A cell is (lower, upper, slope, intercept).
11    cells = [(-inf, inf, [1], [0])]
12
13    for W, b in layers:
14        new_cells = []
15        for lower, upper, slope, intercept in cells:
16            # key observation: in a cell C_prev(l, u, slope, intercept), we have
17            #   z_prev = slope @ x + intercept, so the linear function
18            #   in the current cell C(?, ?, alpha, beta) will be
19            #   z = W @ z_prev + b = (W @ slope) @ x + (W @ intercept + b)
20            alpha = W @ slope
21            beta = W @ intercept + b
22
23            roots = {
24                -beta[j] / alpha[j]
25                for j in range(len(beta))
26                if alpha[j] != 0
27                and lower < -beta[j] / alpha[j] < upper
28            }
29            boundaries = [lower, *sorted(roots), upper]
30
31            for left, right in consecutive_pairs(boundaries):
32                x = choose_point(left, right)
33                active = alpha @ x + beta > 0 # list of booleans
34                new_slope = where(active, alpha, 0)
35                new_intercept = where(active, beta, 0)
36                new_cells.append(
37                    (left, right, new_slope, new_intercept)
38                )
39
40        cells = new_cells
41
42    return cells

```

It discovers the linear cells of a one-dimensional ReLU network, with:

$$\text{NN}(x) = W_L \text{ReLU}(\dots \text{ReLU}(W_1 x + b_1) \dots) + b_L. \quad (11)$$

To prove the correctness algorithms, the key step is to show, given a linear cell of the network up to layer $m - 1$, how to discover the linear cells of the network up to layer m .

As a precondition, we already have a linear cell of the network when we begin: $C(-\infty, \infty, [1], [0])$ represents the entire real line, with slope $[1]$ and intercept $[0]$.

Now, assume at layer $m - 1$ we have a linear cell $C(l, u, \text{slope}, \text{intercept})$ which is correctly discovered. Then, at layer m , as shown in the code comment, we have a linear function:

$$z = W @ z_{m-1} + b = (W @ \text{slope}) @ x + (W @ \text{intercept} + b) \quad (12)$$

Since z is a list of n_m elements, we have to check when $z_j = 0$ for each $j = 1, \dots, n_m$. This gives us the new potential breakpoints.

Now, to determine how the resulting cells are formed, we can take a point within each interval ($\text{breakpoint}_i, \text{breakpoint}_{i+1}$) and check whether the resulting neuron would be active. This yields the new slope and intercept for that cell.

Note that in the actual implementation, the last layer is piecewise quadratic as noted in Remark 1.1, but the reasoning is similar. We just further have to break up the cells based on the breakpoints of the piecewise quadratic function.

3 Multi-dimensional ReLU NN region discovery

We now extend Algorithm 1 to the multi-dimensional input case. The main difference is that the cells are no longer intervals, but convex polyhedra.

Consider a ReLU network

$$\text{NN}(x) = W_L \text{ReLU}(\dots \text{ReLU}(W_1 x + b_1) \dots) + b_L, \quad (13)$$

where now $x \in \mathbb{R}^d$. As before, hidden layers can be multi-dimensional:

$$W_i \in \mathbb{R}^{n_{i+1} \times n_i}, \quad b_i \in \mathbb{R}^{n_{i+1}}.$$

In the one-dimensional case, a cell was represented by

$$\text{C}(l, u, \text{slope}, \text{intercept}).$$

In the multi-dimensional case, we instead represent a cell by

$$\text{C}(A, c, \text{slope}, \text{intercept}),$$

where

$$Ax \leq c$$

describes the current polyhedral region, and on this region we have

$$z_{m-1} = \text{slope } x + \text{intercept}.$$

Here

$$\text{slope} \in \mathbb{R}^{n_{m-1} \times d}, \quad \text{intercept} \in \mathbb{R}^{n_{m-1}}.$$

As a precondition, we already have a linear cell of the network when we begin:

$$\text{C}([], [], I, 0)$$

represents the entire space $\mathbb{R}^d$, with slope I and intercept 0.

Now, assume at layer $m - 1$ we have a linear cell

$$\text{C}(A, c, \text{slope}, \text{intercept})$$

which is correctly discovered. Then, at layer m , we again have a linear function:

$$z = W \otimes z_{m-1} + b = (W \otimes \text{slope}) \otimes x + (W \otimes \text{intercept} + b). \quad (14)$$

Therefore define

$$\alpha = W \otimes \text{slope}, \quad (15)$$

$$\beta = W \otimes \text{intercept} + b. \quad (16)$$

Then, inside the current cell, every pre-activation is of the form

$$z_j = \alpha_j x + \beta_j, \quad (17)$$

where α_j is the j -th row of α .

In the one-dimensional case, the possible new boundaries were points satisfying

$$\alpha_j x + \beta_j = 0.$$

In the multi-dimensional case, the possible new boundaries are hyperplanes:

$$\alpha_j x + \beta_j = 0, \quad j = 1, \dots, n_m. \quad (18)$$

Each such hyperplane splits the current polyhedral cell into two possible pieces:

$$\alpha_j x + \beta_j \geq 0, \quad \text{where neuron } j \text{ is active,} \quad (19)$$

$$\alpha_j x + \beta_j \leq 0, \quad \text{where neuron } j \text{ is inactive.} \quad (20)$$

Thus, instead of sorting roots on the real line, we refine the current region by intersecting it with all possible activation constraints. Equivalently, for each activation pattern

$$s \in \{0, 1\}^{n_m},$$

we construct the candidate region

$$R_s = \{x : Ax \leq c, \quad \alpha_j x + \beta_j \geq 0 \text{ if } s_j = 1, \quad \alpha_j x + \beta_j \leq 0 \text{ if } s_j = 0\}. \quad (21)$$

If R_s is non-empty, then it is a new linear cell. On this cell, the ReLU activation is fixed, so we get

$$\text{new_slope} = \text{where}(s, \alpha, 0), \quad (22)$$

$$\text{new_intercept} = \text{where}(s, \beta, 0). \quad (23)$$

This is the exact multi-dimensional analogue of the one-dimensional update.

The corresponding algorithm is:

Listing 2: Discover multi-dimensional linear regions of a ReLU network

```

1 def discover_regions(layers, d):
2     """
3     The network has a d-dimensional input. Hidden layers can be multi-dimensional:
4     >>> layers[i] = (W_i, b_i)
5     >>> W_i.shape = (n_{i+1}, n_i)
6     >>> b_i.shape = (n_{i+1},)
7     """
8
9     # A cell is (A, c, slope, intercept), where
10    #   A @ x <= c
11    # A.shape = (num_constraints, d)
12    # slope.shape = (num_constraints, d)
13    # and inside this region
14    #   z = slope @ x + intercept.
15    cells = [([], [], I_d, 0)]
16
17    for W, b in layers:
18        new_cells = []
19        for A, c, slope, intercept in cells:
20            # key observation: in a cell C(A, c, slope, intercept), we have
21            #   z_prev = slope @ x + intercept, so the linear function
22            # in the current cell C(?, ?, alpha, beta) will be
23            #   z = W @ z_prev + b = (W @ slope) @ x + (W @ intercept + b)
24            alpha = W @ slope
25            beta = W @ intercept + b
26
27            for active in all_boolean_patterns(len(beta)):
28                A_new = list(A)
29                c_new = list(c)
30
31                for j in range(len(beta)):
32                    if active[j]:
33                        # alpha[j] @ x + beta[j] >= 0
34                        # equivalently: -alpha[j] @ x <= beta[j]
35                        A_new.append(-alpha[j])
36                        c_new.append(beta[j])
37                    else:
38                        # alpha[j] @ x + beta[j] <= 0
39                        # equivalently: alpha[j] @ x <= -beta[j]
40                        A_new.append(alpha[j])
41                        c_new.append(-beta[j])
42
43                    if is_nonempty_polyhedron(A_new, c_new):
44                        new_slope = where(active, alpha, 0)
45                        new_intercept = where(active, beta, 0)
46                        new_cells.append(
47                            (A_new, c_new, new_slope, new_intercept)
48                        )
49
50        cells = new_cells
51
52    return cells

```

To prove the correctness of the algorithm, the key step is again to show, given a linear cell of the network up to layer $m - 1$, how to discover the linear cells of the network up to layer m .

Assume that at layer $m - 1$ we have correctly discovered a cell

$$C(A, c, \text{slope}, \text{intercept}).$$

By correctness of this cell, for all x satisfying $Ax \leq c$, we have

$$z_{m-1} = \text{slope } x + \text{intercept}.$$

After applying the affine map of layer m , we obtain

$$z = Wz_{m-1} + b = (W\otimes \text{slope})x + (W\otimes \text{intercept} + b) = \alpha x + \beta.$$

Therefore, inside the current cell, the sign of each neuron is determined by the affine inequality

$$\alpha_j x + \beta_j \gtrless 0.$$

The ReLU is linear exactly on regions where these signs are fixed. Hence, the current cell must be split according to all possible choices of signs of

$$\alpha_1 x + \beta_1, \dots, \alpha_{n_m} x + \beta_{n_m}.$$

For a fixed activation pattern s , the region R_s is precisely the set of points in the old cell which realize that activation pattern. If this region is empty, then that pattern never occurs in the old cell and should be discarded. If it is non-empty, then the ReLU output on this region is

$$\text{ReLU}(z)_j = \begin{cases} \alpha_j x + \beta_j, & s_j = 1, \\ 0, & s_j = 0. \end{cases}$$

Thus, on R_s , the new representation is exactly

$$\text{new_slope} = \text{where}(s, \alpha, 0), \tag{24}$$

$$\text{new_intercept} = \text{where}(s, \beta, 0). \tag{25}$$

Therefore every non-empty region generated by the algorithm is a correct linear cell for the network up to layer m . Conversely, any true linear cell of the network up to layer m must lie inside some previously discovered cell at layer $m - 1$, and must have a fixed activation pattern at layer m . Hence it is contained in one of the non-empty regions R_s constructed above. This proves that the algorithm discovers exactly the linear cells of the network layer by layer.

Note that, just as in the one-dimensional case, if the last layer is piecewise quadratic as noted in Remark 1.1, the same reasoning applies. The only difference is that instead of further splitting intervals by the breakpoints of the piecewise quadratic function, we further split polyhedral cells by the hypersurfaces on which the corresponding piecewise quadratic expression changes.

4 2D SDE verification example

5 Certificate training

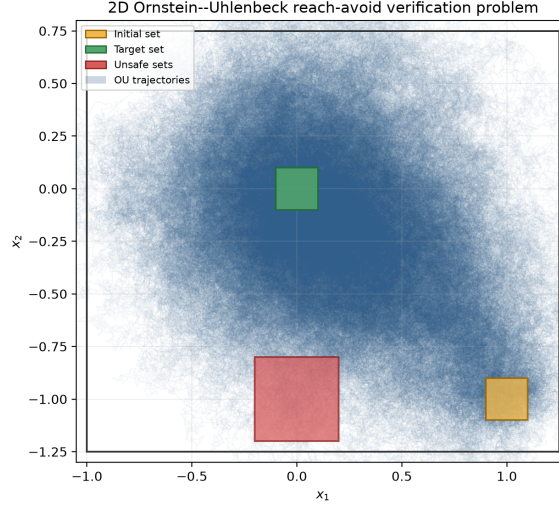


Figure 1: Verification of a piecewise-quadratic certificate for the two-dimensional Ornstein-Uhlenbeck process.

Before Greg gave feedback on my work, I was working on training a piecewise-quadratic certificate for a problem of an Ornstein-Uhlenbeck process in 2D, as seen on Figure 1.

I was training the certificate with the losses:

1. **Boundary loss.** On sampled points from the initial and unsafe sets, including all corners of their hyperrectangles, I penalized $\max\{V(x) - (\alpha - \delta), 0\}$ for $x \in X_0$ and $\max\{(\beta + \delta) - V(x), 0\}$ for $x \in X_u$, where the constraint margin was $\delta = 0.1$.
2. **Generator loss.** For sampled domain points outside the target, I penalized the cheaper of the two violations

$$\max\{\mathcal{L}V(x) + \varepsilon + \delta_{\mathcal{L}}, 0\} \quad \text{and} \quad \max\{\beta + \delta - V(x), 0\},$$

with generator margin $\delta_{\mathcal{L}} = 0.02$. This reflects the disjunctive condition that either $x \notin \{V \leq \beta\}$ or the generator must be sufficiently negative. This loss was enabled after 50 epochs of boundary pretraining.

3. **Concavity loss.** For sampled pairs x, y from the same hyperrectangle, I penalized violations of midpoint concavity,

$$\max\left\{\frac{V(x) + V(y)}{2} - V\left(\frac{x + y}{2}\right), 0\right\}.$$

4. **Regularization.** I added a small mean-squared parameter penalty to discourage unnecessarily large network weights.

I verified it with an approximate verifier (that samples points instead of checking the entire region) and it found some violations of the certificate conditions, as seen on Figure 2.

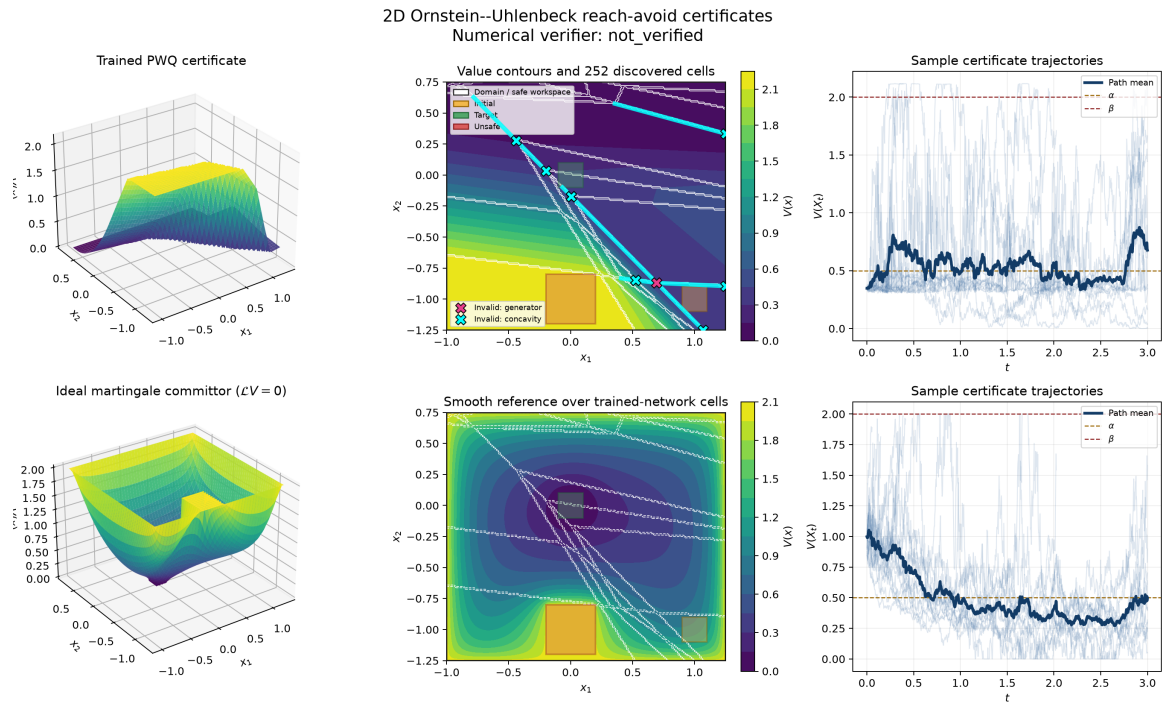


Figure 2: Comparison of the trained piecewise-quadratic certificate.